Variables y tipos de dato

Lección 3 — 1MAK, SST-INF-01c — Qué es un programa (repaso), variables, char/int/float

29/09/2026

1 Repaso: qué es un programa

Definición

Un **programa** (*programma*) es, básicamente, una **lista ordenada de instrucciones** (*lista ordinata di istruzioni*) que le damos a nuestra máquina: una lista ordenada de operaciones que hay que hacer, ejecutada **un paso a la vez**.

Ejemplo

La comparación con la receta: una **receta** es prácticamente un programa, porque el **orden** de las operaciones importa. Si hago una operación antes que otra, cambia el resultado final. La **lista de compras**, en cambio, **no** es un programa: si compro la carne antes que la pasta, el resultado final (las compras hechas) no cambia. El orden en que compro cada cosa no tiene importancia.

Nota

Un programa, al final, solo hace **operaciones**: hace **cálculos**, toma un objeto (un dato) y lo transforma en otro.

1.1 Ejemplo práctico en el aula: el área del cuadrado

Ejemplo

El profesor le propone a la clase: *"calcula el área del cuadrado de lado 5"*. Un alumno, con el número 5 "en la mano", responde correctamente: **multiplica** 5 por 5 y obtiene 25.
Pregunta del profesor: este **5** que se le dio al alumno, ¿dónde se guarda? **En la memoria**. ¿Y el programa, dónde guarda sus datos? También **en la memoria**. Entonces hace falta crear una especie de **caja virtual** donde poner el número que hay que **recordar**.

Ejemplo

Las cajas "Carlo" y "Giuseppe" (ejemplo hecho en el pizarrón): el profesor crea una primera caja, la llama **Carlo** y pone adentro el número 5. Después crea una segunda caja, que llama **Giuseppe**, y en esta caja pone el **resultado** de **Carlo multiplicado por Carlo** (el **cuadrado**: lado por lado).
Si en Carlo está el número 5, en Giuseppe termina el número 25. Si en Carlo se pone el número 2, en Giuseppe termina el número 4.
Esta es, en resumen, la idea de **variable** (*variabile*): una caja con un nombre, que contiene

un valor que el programa puede usar y **cambiar**.

2 Las variables

Definición

Las "cajas" en la memoria de la computadora, los cajones donde se guarda la información, se llaman **variables**. Una **variable** es un lugar donde se guarda una **información**. Las variables pueden ser **más o menos grandes**, según el número (o el **tipo de dato**) que tienen que contener.

2.1 Cuánta memoria tiene una computadora: de los terabytes a los bytes

Ejemplo

Pregunta en el aula: ¿cuánta **memoria** (no RAM, sino el almacenamiento total) tiene la computadora de tu casa? Respuesta de la clase: más o menos **un terabyte (TB)**.
El profesor guía a la clase en la conversión, paso a paso. Recuerda que cada vez que se pasa de una unidad de medida a la siguiente se agregan o se quitan **siempre tres ceros**:

1 **TB**	=	1.000 **GB** (gigabyte)
1.000 GB	=	1.000.000 **MB** (megabyte)
1.000.000 MB	=	1.000.000.000 **KB** (kilobyte)
1.000.000.000 KB	=	1.000.000.000.000 **B** (byte)

Entonces un terabyte corresponde a **un billón de bytes** (mil miles de millones; en italiano *mille miliardi*): un billón de "cajones", y cada uno puede **contener un número** que va, como vamos a ver, de 0 a 255.

Nota

Para convertir entre una unidad de medida y la siguiente/anterior: si vamos hacia arriba (de byte hacia terabyte) se **quitan ceros** y se cambia la sigla; si vamos hacia abajo se **agregan ceros**. El **factor** es siempre 1.000 (tres ceros) entre una unidad y la siguiente.

3 El tipo `char`: la variable más pequeña

Definición

El **byte** es la **unidad básica** de memoria: está formado por **8 bits**. El tipo de variable más pequeño que se puede crear en un programa se llama **char**, y está hecho de **8 bits (1 byte)**. Con 8 bits se pueden representar $2^8 = 256$ **combinaciones** distintas. Por eso una variable `char` puede guardar los números enteros que van de **0 a 255**.

Atención

¿Por qué 255 y no 256? Porque si hay 256 combinaciones posibles, y el **0** es una de esas combinaciones (representa un número válido, no "el vacío"), el número **más alto** que se puede alcanzar es 255, no 256. Si hay que guardar el número **300**, una variable `char` ya no alcanza: hace falta un **tipo más grande**.

3.1 Repaso: combinaciones y sistema binario

Ejemplo

Con dos lamparitas: ¿cuántas **combinaciones** (encendida/apagada) son posibles? **Cuatro**: apagada-apagada, apagada-encendida, encendida-apagada, encendida-encendida. Recordando la **codificación binaria** de la clase anterior, a cada combinación le corresponde un número:

$$\begin{array}{ll} \text{apagado-apagado} & = 0 \\ \text{apagado-encendido} & = 1 \\ \text{encendido-apagado} & = 2 \\ \text{encendido-encendido} & = 3 \end{array}$$

¿Por qué esta correspondencia? Por la misma regla de conversión decimal→binario de la clase anterior: **dividir varias veces por 2** y leer los restos **de abajo hacia arriba**. Por ejemplo, para convertir el número **2**: $2 \div 2 = 1$ con resto 0; $1 \div 2 = 0$ con resto 1. Leyendo los restos de abajo hacia arriba se obtiene **10**, es decir el 2 en binario.

Ejemplo

Con tres lamparitas (3 bits), las **combinaciones posibles** suben a $2^3 = 8$:

$$\begin{array}{ll} 000 & 0 \\ 001 & 1 \\ 010 & 2 \\ 011 & 3 \\ 100 & 4 \\ 101 & 5 \\ 110 & 6 \\ 111 & 7 \end{array}$$

Con **8 bits** (un char), las combinaciones posibles son $2^8 =$ **256**: se va de 0 hasta $256-1 =$ **255**.

3.2 Declarar una variable char

Ejemplo

```
char michele;
```

Esta línea **crea una variable** de tipo `char`, llamada `michele`, que puede contener un **número entero** entre 0 y 255.

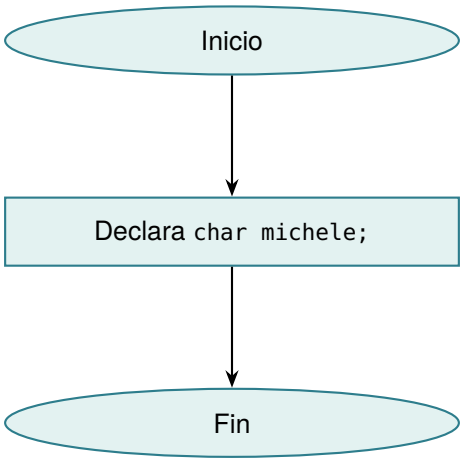


Figura 1: Diagrama de flujo: declaración de una variable `char`.

4 El tipo `int`

Definición

Existe otro tipo de variable, llamado **int** (*integer*, entero; en italiano *intero*): ocupa, como mínimo, **16 bits**, es decir **2 bytes**. Con 16 bits se pueden representar $2^{16} = 65.536$ combinaciones. Por eso una variable **`int`** puede contener números que van de **0 a 65.535**.

Ejemplo

Una variable `int` puede contener números **mucho más grandes** que un `char`: por ejemplo el número de habitantes de Friuli (la región donde estamos), la distancia en metros entre dos ciudades, la altura en metros de una montaña. Cualquier valor que esté entre 0 y 65.535.

4.1 El tamaño de int depende de la máquina

Atención

El **tamaño** de la variable `int` **no es siempre igual**: depende del **tipo de máquina** (de la tecnología del hardware) en la que funciona el programa.

- En un **Arduino**, que tiene un microcontrolador de **8 bits**, el `int` es igualmente de **16 bits** (el mínimo que garantiza el lenguaje C).
- En una máquina con un procesador de **16 bits**, el `int` sigue siendo de **16 bits**.
- En máquinas más potentes, de **32 o 64 bits** (por ejemplo una computadora con Windows de 64 bits), el `int` toma exactamente el **ancho de la máquina**: entonces pasa a ser de 32 o de 64 bits.

En resumen: el tamaño de la "caja" `int` depende de la tecnología con la que está construida la máquina. El mínimo garantizado es siempre **16 bits**.

4.2 La escala de los números: de 0 a 65.535, y la versión con signo

Ejemplo

Dibujo en el pizarrón: el profesor dibuja una escala numérica de **0** a **65.535** y le pide a la clase que encuentre el **punto medio**. La clase encuentra correctamente más o menos **32.768** como punto medio, y más o menos **16.000** como **un cuarto** de la escala.

Nota

En realidad, el lenguaje C normalmente **no empieza desde 0**. La escala tiene el mismo ancho (siempre 65.536 números en total), pero en lugar de empezar en 0, el **0** se pone en la **mitad de la escala**. Hacia la derecha se sube hasta **+32.767**; hacia la izquierda (números negativos) se baja hasta **-32.768**. Así, con una variable `int`, se pueden representar números tanto **positivos** como **negativos**.

Definición

Resumen de los **rangos** (*range*) de las variables que vimos hasta ahora:

- **char**: de **0** a **255** (**solo positivos**).
- **int**: de **-32.768** a **+32.767**.

¿Por qué los dos extremos no son iguales en valor absoluto? Porque el **cero** existe **una sola vez** (no hay un "más cero" y un "menos cero" distintos). Entonces a un lado de la escala (los positivos) hay que **quitarle una unidad** respecto al otro lado (los negativos), para no contar **dos veces** el cero.

Ejemplo

```
int paolo;
```

Esta línea **crea una variable** de tipo `int`, llamada `paolo`.

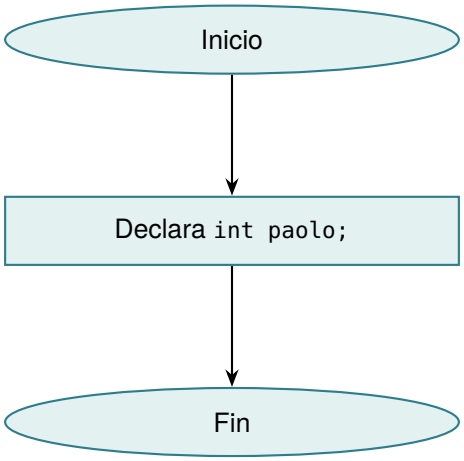


Figura 2: Diagrama de flujo: declaración de una variable `int`.

5 Números sin coma y números con coma

Atención

Tanto el tipo `char` como el tipo `int` contienen **solo números enteros, sin coma** (*senza virgola*). No se puede guardar el número 7,5 en una variable `int`.
Si intentamos poner el número **32,9** en una variable `int`, el valor guardado va a ser simplemente **32**: la parte decimal se **pierde**, **no se redondea**. Lo mismo pasa con 32,99999999: el resultado guardado sigue siendo 32. Si no se tiene en cuenta, este comportamiento puede **acumular errores** con el tiempo y convertirse en un problema serio en un programa.

6 El tipo `float`

Definición

Para guardar números **con coma** (*con la virgola*) existe un **tercer tipo** de variable, llamado **float**. Ocupa **32 bits**, es decir **4 bytes**.

Atención

Aclaración (no se dijo explícitamente en clase, se verificó después): el **rango real** de un `float` de 32 bits **no** es "más o menos ± 2 mil millones" (ese valor es correcto, en cambio, para un `int`/`long` de 32 bits). El rango real de un `float` es **mucho más amplio**: más o menos $\pm 3,4 \times 10^{38}$. Esto pasa porque un número en **coma flotante** no usa todos sus bits para contar en secuencia como un entero. Los divide entre signo, **exponente** y **mantisa** (parte decimal), un poco como la **notación científica**. Así puede representar tanto números enormes como números pequeñísimos, pero con una **precisión limitada** en las cifras significativas.

Ejemplo

Ejemplo en el aula: ¿se puede guardar en un `float` un número como 322.423,7411? Sí, un `float` puede contener **números con coma** de este tipo.

6.1 Por qué elegir el tipo correcto: la comparación con los cajones

Ejemplo

Comparación con el armario: un armario grande tiene muchos **cajones**, algunos grandes y otros pequeños. Si tenemos un cajón muy grande y ponemos adentro solo un par de medias (porque no tenemos otras), es un **desperdicio de espacio**.
De la misma manera, en un programa: si hay que manejar **datos pequeños**, no tiene sentido ocupar más espacio de memoria del necesario usando siempre el **tipo más grande**.

Ejemplo

Por qué importa, en la práctica: si un programa crea no una, sino **100.000 variables**, y para cada una bastaría un solo `byte` (`char`) en lugar de 4 bytes (`float`), la **diferencia** es de **300.000 bytes** ocupados sin necesidad.
Ejemplo de los videojuegos: supongamos que un videojuego necesita una **tarjeta de video** con 64 GB de RAM (ya es un gasto importante). Si el programador, sin pensar en estos problemas, usara siempre `float` incluso donde no hace falta, la memoria necesaria podría subir hasta 500 GB de RAM. Una tarjeta de video que hoy cuesta 1.000 euros costaría entonces 15.000: la mayoría de las personas **no podría jugar**, y el programador no vendería el juego. Por eso al programador **le conviene** que el espacio ocupado por el programa sea **lo más bajo posible**, para que el juego funcione también en hardware menos potente y más barato.

Definición

Regla práctica para elegir el tipo de variable:

- Si el número que hay que guardar **tiene coma**, hace falta **siempre** el `float`, aunque el número sea pequeño (por ejemplo entre 1 y 5).
- Si el número **no tiene coma** y es pequeño (por ejemplo cerca de 200), alcanza con un **`char`**: se ahorra memoria respecto a un `int` o un `float`.
- Si el número no tiene coma pero es más grande que 255, hace falta un **`int`**.

Nota

Estos tres tipos de variables hay que **aprenderlos de memoria**: para cada uno hay que saber cuántos bits ocupa (8, 16, 32) y el **rango de números** que puede representar. No basta con reconocerlos: hay que estudiarlos.

7 Cómo se crea (declara) una variable

Definición

Para **crear una variable** en C se escriben, en este orden:

1. el **tipo** de la variable (`char`, `int` o `float`);
2. un **espacio**;
3. el **nombre** elegido para la variable;
4. un **punto y coma** (`;`, en italiano *punto e virgola*) que cierra la instrucción.

Ejemplo

Ejemplos escritos en el pizarrón durante la clase:

```
int paolo;
```

```
char michele;
```

El **nombre elegido** no tiene que ser necesariamente un nombre de persona: una variable se puede llamar `lampadina` (lamparita), `sasso` (piedra), `zqw`, o simplemente `x`. Cualquier nombre, siempre que respete las **reglas** que vemos a continuación.

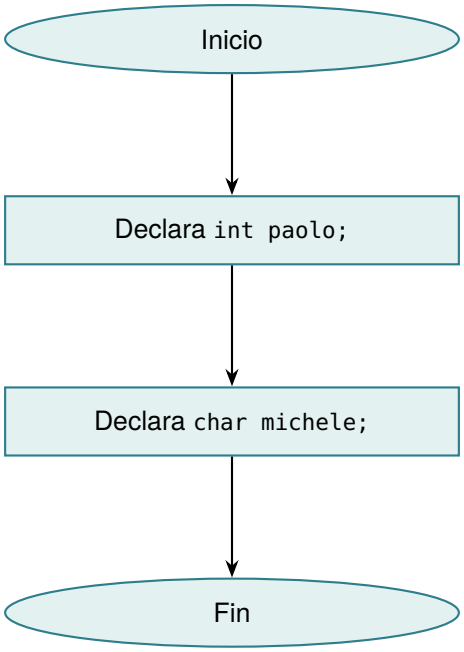


Figura 3: Diagrama de flujo: declaración en secuencia de dos variables de tipo distinto.

8 Las reglas para el nombre de una variable

Atención

No se le puede dar a una variable **cualquier** nombre: el nombre de una variable tiene que respetar **tres reglas** precisas.

8.1 Regla 1: no puede empezar con un número

Ejemplo

```
int 7zq;   // NO valido: empieza con un numero
int zq7;   // valido: el numero no esta al inicio
```

Lo importante es que el nombre **no empiece** con una **cifra**: un número puede aparecer en cualquier otra posición del nombre, incluso justo después de la primera letra.

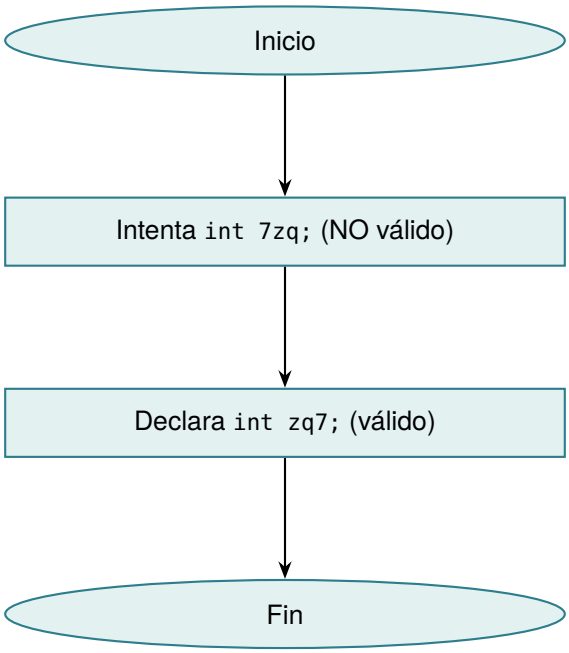


Figura 4: Diagrama de flujo: comparación entre un nombre de variable no válido y uno válido.

8.2 Regla 2: no puede contener símbolos especiales

Definición

El nombre de una variable puede contener solo **letras, números y guion bajo** (*underscore*, el guion bajo _). En el teclado italiano normalmente se escribe con la tecla a la derecha del punto, manteniendo apretada la tecla de mayúsculas. No se acepta **ningún otro símbolo especial**: ni +, $, €, /, \, comillas, espacios, signos de pregunta, ni ningún otro carácter que no sea letra o número.

Ejemplo

Ejemplos escritos en el pizarrón (válidos y no válidos):

```
int x+2;          // NO valido: contiene el simbolo '+'
int _x2;          // valido: el guion bajo esta permitido
int Ciao Mauro;   // NO valido: contiene un espacio
int Giochi?;      // NO valido: contiene '?'
```

8.3 Regla 3: es case sensitive

Definición

El C es **case sensitive** (distingue mayúsculas y minúsculas): las **mayúsculas y las minúsculas** se consideran **caracteres distintos**. Por eso, dos variables que se diferencian solo por mayúsculas/minúsculas son dos variables **completamente distintas** (dos "cajas" distintas), aunque se pronuncien igual.

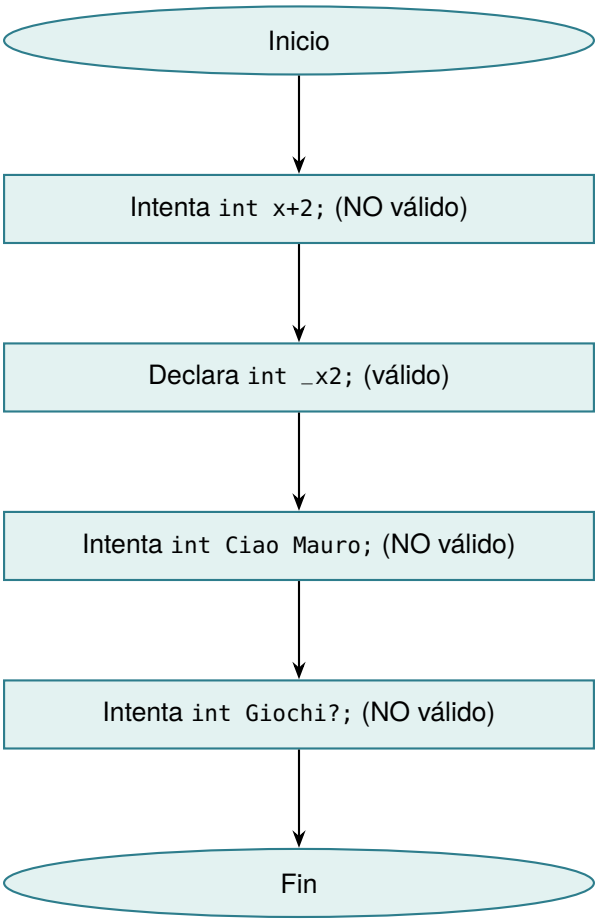


Figura 5: Diagrama de flujo: cuatro intentos de declaración, con símbolos válidos y no válidos en el nombre.

Atención

Ejemplo en el aula: una variable llamada `y` (minúscula) y una llamada `Y` (mayúscula) son **dos variables distintas**, aunque en voz alta se lean igual ("i griega"; en italiano *"i greca"*). Hay que tener **mucho cuidado** con esta diferencia: confundir las dos puede crear **errores difíciles** de encontrar en un programa.

Ejemplo

Último ejemplo, que quedó pendiente al final de la clase: **dos variables distintas**, en las que se puede poner por ejemplo el número 42 en una y 73 en la otra. Siempre respetando las tres reglas vistas: **no empezar con un número** y **no usar símbolos** no permitidos.

9 Qué recordar de esta lección

- Un **programa** es una **lista ordenada de instrucciones** que se ejecutan un paso a la vez; **el orden importa** (a diferencia de una lista de compras).
- Una **variable** es una "caja" en la **memoria** donde se guarda un dato, identificada por un **nombre**.

- **char**: **8 bits** (1 byte), números enteros de **0 a 255**.
- **int**: **mínimo 16 bits** (2 bytes, depende de la máquina), números enteros de **-32.768 a +32.767**.
- **float**: **32 bits** (4 bytes), números **con coma**, con un **rango mucho más amplio** de lo que haría pensar su tamaño en bits.
- Elegir siempre el tipo de variable **más pequeño posible** que alcance para el dato que hay que guardar, para no **desperdiciar memoria**.
- El nombre de una variable: no puede **empezar con un número**, puede contener solo letras/números/guion bajo, y es **case sensitive**.

10 Glosario: palabras en italiano que se usan en clase

Italiano	Español
variabile	variable
tipo di dato	tipo de dato
scatola	caja
cassetto	cajón
memoria	memoria
dichiarare / dichiarazione	declarar / declaración
intero (int)	entero
numero con la virgola (float)	número con coma (decimal)
virgola	coma
punto e virgola	punto y coma
nome della variabile	nombre de la variable
maiuscolo / minuscolo	mayúscula / minúscula
underscore (trattino basso)	guion bajo
diagramma di flusso	diagrama de flujo
inizio / fine	inicio / fin
valido / non valido	válido / no válido
spreco	desperdicio
lavagna	pizarrón